PROYECTO FIN DE CICLO

IES JOSÉ LUIS MARTÍNEZ PALOMO

Departamento de Informática y Comunicaciones
Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

APP DE FICHAJE CON GEOLOCALIZACIÓN Y REGISTRO DE TAREAS

Autor: **Manuel Andrés de Larrea Gil**
Tutor: **Jesús Ibáñez López**

Alquerías, 4 de diciembre de 2025

Índice

1. Objetivo y análisis de idoneidad 3
1.1 Utilidad y beneficios 3
1.2 Estudio de aplicaciones similares 3
2. Planificación y viabilidad 4
2.1 Coste económico 5
2.2 Reflexión del proyecto: posibilidades, alcance, limitaciones y mejoras 5
3. Análisis de información y desarrollo 6
3.1 Cuándo 6
3.2 De dónde se obtiene o quién la ofrece: 7
3.3 De qué forma se obtiene: 7
3.4 Lista de los apartados básicos a desarrollar 7
4. Diseño 8
4.1. Modelo Entidad Relación (modelo conceptual) 8
4.2. Modelo estándar de datos 9
4.3. Diseño de la interfaz 11
4.4. Tecnologías utilizadas y justificación 14
5. implementación 16
5.1. Entorno de desarrollo y ejecución 16
6. Prueba 20
7. Mantenimiento 21
8.Conclusiones 22
9.Bibliografía 23

1. OBJETIVO Y ANÁLISIS DE IDONEIDAD

El objetivo del proyecto es desarrollar una aplicación móvil que permita a los trabajadores registrar sus fichajes de entrada y salida de manera rápida mediante la geolocalización del móvil personal del trabajador. La app validará que el empleado se encuentre dentro del trabajo y si es así podrá comenzar el fichaje, registrará las áreas de trabajo visitadas y permitirá seleccionar las tareas realizadas en cada área para llevar un registro diario de tareas y áreas visitadas.

1.1 UTILIDAD Y BENEFICIOS

La aplicación permite que los trabajadores puedan fichar al entrar y al salir del trabajo de la manera más ágil y sencilla posible gracias a la geolocalización. Esto significa que no es necesario que el trabajador pierda tiempo rellenando hojas de asistencia ni que exista una supervisión física constante, ya que el sistema valida automáticamente la presencia del empleado dentro de la zona de trabajo definida.

Por otra parte, el registro de actividades es tan fácil como seleccionar las casillas correspondientes una vez terminado el día de trabajo. Esta funcionalidad reduce la posibilidad de errores en la asignación de tareas y permite que cada empleado tenga un control claro de lo que ha realizado, simplificando la gestión personal de su jornada laboral.

Por otro lado, a la empresa le facilita y agiliza muchas tareas relacionadas con la administración de los trabajadores. La aplicación proporciona un seguimiento diario con validación geográfica, lo que asegura que los fichajes se han realizado efectivamente en las zonas correctas. Además, el registro de las tareas que han ido realizando los empleados cada día trabajado permite generar informes precisos y rápidos sobre la productividad, el tiempo dedicado a cada área o actividad, y el cumplimiento de las funciones asignadas. Esto, a su vez, mejora la toma de decisiones, la planificación de recursos y la optimización de los procesos internos de la empresa.

1.2 ESTUDIO DE APLICACIONES SIMILARES

Las principales aplicaciones del mercado que usan geolocalización para el control de presencia son las siguientes:

- **aTurnos:** Permite definir zonas de trabajo con un radio configurable y valida la ubicación del empleado antes de permitir el fichaje. Incluye gestión de turnos y calendarios.
- **Jibble:** Utiliza geovallas (geofencing) para crear límites virtuales. Ofrece tracking en tiempo real y reconocimiento facial opcional.
- **Kronos:** Sistema empresarial con geofencing avanzado, cumplimiento RGPD y múltiples métodos de fichaje.
- **Time Laboris:** Aplicación española con control por GPS y generación de informes para cumplir con la normativa laboral.

Éstas aplicaciones del mercado se centran en seguir la ruta del trabajador de manera exhaustiva para tenerlo controlado.

A diferencia de muchas soluciones comerciales, que están muy orientadas al control estricto del trabajador, esta aplicación pretende ser más práctica y menos invasiva. La idea es que registre de forma automática las zonas por las que pasa el empleado y las tareas que realiza, sin necesidad de estar monitorizando cada movimiento.

Planificación proyecto Fin de Grado

FASES DEL PROYECTO	SEMANAS	TAREAS
Análisis y planificación	1º Del 06 al 10/10/25	Concretar proyecto, tecnologías a usar y usos de la app Objetivo y análisis de ideonidad Planificación y viabilidad Análisis de información y desarrollo
	2º Del 13 al 17/10/25	Creación del documento sobre el seguimiento del proyecto. Incluye hasta el modelo Entidad-Relación
Diseño	3º y 4º Del 20 al 31/10/25	Modelo Entidad Relación Tecnologías y justificación Comienzo del diseño de la app Diseño de interfaz Diagrama de Clases
Implementación	5º y 6º Del 03 al 14/11/25	Desarrollo y código Elaboración del prototipo Ejemplos y test
Prueba y mantenimiento	De la 7º a la última Del 17/11 al 04/12/25	Pruebas de la app, correcciones y mejoras

2. PLANIFICACIÓN Y VIABILIDAD

Debido a que en la segunda entrega (15 de noviembre) ya debe de estar prácticamente terminado el proyecto, la implementación se ha reducido a 2 semanas. Pero siendo objetivos, llevará algo más de tiempo, por lo que usaremos la última fase del proyecto.

Tras la última entrega, hay que tener en cuenta la elaboración de la exposición del proyecto, el cual conlleva la realización de un PowerPoint y la demostración del uso de la app y, por último, la propia exposición.

2.1 COSTE ECONÓMICO

Usaremos programas y licencias gratuitas así que los costes económicos se reducen al propio ordenador que usamos para crear la app (**800 euros,** aproximadamente). El coste de las horas de trabajo de la persona encargada del desarrollo va a ser a tiempo parcial durante tres meses, por lo que se estima un coste de **2500 euros**. En cuanto al teléfono móvil que se usará para las pruebas de la aplicación supondrá un coste de **200 euros**. En total, sale un coste económico de **3500 euros** aproximadamente**.**

Estos costes se amortizarían al comercializar la app teniendo como objetivo a grandes empresas que requieran de un mantenimiento de la aplicación constante. Según el estudio de mercado, se ha podido comprobar que las empresas pagan el mantenimiento teniendo en cuenta el número de empleados. Así que supongamos que se captan:

- 10 empresas pequeñas de 10 empleados.
- Precio medio: 25 €/mes.
- Total mensual: 250 €/mes = 3.000 €/año.

Por tanto, la amortización dependerá realmente de las empresas a las que podamos vender el producto y el tipo de empresas que captemos. Objetivamente, se podría amortizar la app en algo más de un año de mantenimiento, teniendo en cuenta que el tiempo que el desarrollador usará en las empresas será mínimo.

2.2 REFLEXIÓN DEL PROYECTO: POSIBILIDADES, ALCANCE, LIMITACIONES Y MEJORAS

Teniendo en cuenta la formación y la cantidad de recursos en internet, la posibilidad de realizar este proyecto en el tiempo estimado es bastante coherente, pero debemos tener en cuenta que la interfaz gráfica es de lo más importante a la hora de comercializar, por lo que requiere más tiempo y esfuerzo.

La idea del proyecto se debe a mi paso por diferentes empresas (mayormente agrícolas) en las que había un descontento en los trabajadores por usar aplicaciones diferentes para fichar y luego para repartir las horas y tareas. Esta aplicación solucionaría todos esos inconvenientes que surgen a la hora de comprobar si has fichado o no (ya que lo verías en tu móvil directamente con un mensaje), recordar cuánto tiempo has estado en cada sitio y también a reducir las tareas al

sitio en concreto. Todo esto hace una aplicación muy atractiva para empresarios agrícolas que por ejemplo tengan varios invernaderos en un terreno con diferentes plantaciones.

Actualmente las funcionalidades de la aplicación son las siguientes:

- Login de empleados mediante DNI
- Fichaje de entrada validando ubicación GPS dentro de zona de trabajo
- Tracking automático de áreas visitadas durante la jornada
- Selección de tareas realizadas en cada área
- Fichaje de salida con resumen de la jornada

Funcionalidades necesarias:

- Login con contraseña.
- Crear una interfaz para un usuario administrador con posibilidad de añadir empleados, tareas nuevas, áreas nuevas y modificar el horario de los empleados (por si algo ha fallado a la hora de registrar).
- Que los usuarios puedan modificar las tareas realizadas en la vista del informe por si se han equivocado al seleccionar.
- Una base de datos externa a la aplicación. Esto es necesario tanto para la correcta funcionalidad de la app como para la administración de los datos. Manejar las BBDD con Visual Studio creando ETLs y SQL server para las consultas y manejo de datos sería perfecto.
- La posibilidad de fichar mediante NFC con el móvil del usuario para dar otra funcionalidad alternativa a la aplicación.

Tras la finalización del proyecto, el correcto funcionamiento de todas sus funcionalidades y las mejoras necesarias sería necesario también, una actualización de la interfaz para que sea lo más atractiva posible a la hora de su comercialización exitosa

3. ANÁLISIS DE INFORMACIÓN Y DESARROLLO

El presente trabajo va a partir de diferentes puntos a la hora de extraer la información: cuándo, dónde y de qué forma se obtiene.

3.1 Cuándo

La información necesaria para desarrollar la aplicación se va a obtener principalmente en las primeras fases del proyecto, especialmente en la que corresponde al diseño, aunque seguiremos recopilando información durante las pruebas y el desarrollo final. A medida que se implementan los módulos de fichaje (GPS), se realizan pruebas en distintos dispositivos móviles para ajustar la precisión y facilidad de uso.

3.2 De dónde se obtiene o quién la ofrece:

Obtendremos la información de diversas fuentes como las oficiales de Android para ayudarnos con la implementación de las diversas tecnologías como la geolocalización, el tracking, la aplicación con servicios en segundo plano o el manejo de permisos; fuentes académicas como recursos del curso DAM, especialmente de la asignatura Programación en Tecnologías Móvil; de mi tutor Jesús Ibáñez López, para orientación y resolución de dudas; y también de búsqueda en foros y diversas páginas de internet.

3.3 De qué forma se obtiene:

A través de la investigación online, pruebas reales con dispositivos Android y recopilación de experiencias previas en entornos laborales. También mediante el uso de foros técnicos y la comparación de resultados obtenidos al probar el método de fichaje, la correcta geolocalización y la correcta selección de las tareas según las áreas visitadas.

3.4 lista de los apartados básicos a desarrollar

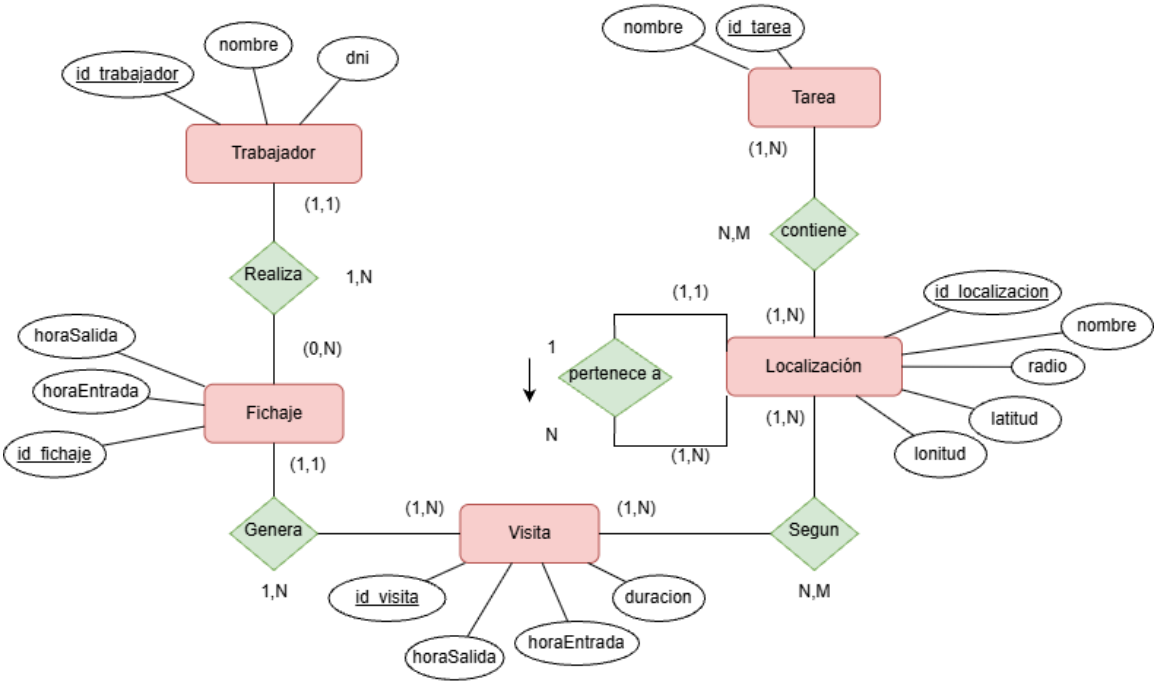
Prioridad	Módulo o componente	Descripción
Alta	**Gestión de usuarios y autenticación**	Permitir que cada trabajador se identifique antes de fichar, asegurando la validez del registro.
Alta	**Fichaje mediante GPS**	Registrar automáticamente la ubicación del empleado dentro del área de trabajo.
Alta	**Detección correcta del área de trabajo**	La selección del área distinguiendo lo que es la Zona de trabajo (todo el recinto del trabajo) y el Área de trabajo (área específica que le corresponden tareas específicas)
Alta	**Registro de tiempos y tareas**	Calcular el tiempo total trabajado y permitir al usuario seleccionar las tareas realizadas antes de cerrar el fichaje.
Media	**Historial de fichajes y estadísticas**	Mostrar al usuario un historial con sus fichajes anteriores y el tiempo trabajado.
Baja	**Notificaciones automáticas**	Enviar avisos si el trabajador no ha fichado o si abandona el área de trabajo sin hacerlo.
Baja (mejora futura)	**Permiso de administración para la empresa**	Permitir a los responsables ver los fichajes y controlar las horas trabajadas por empleado, que puedan modificar tareas e incluir roles.

4. DISEÑO

4.1. Modelo Entidad Relación (modelo conceptual)

El modelo conceptual de la base de datos incluye las siguientes entidades principales:

- **Empleado**: Representa a cada trabajador con su identificación (DNI) y nombre.
- **Localización**: Representa tanto las zonas de trabajo (radio grande, ~300m) como las áreas específicas dentro de cada zona (radio pequeño, ~15-50m). Utiliza una relación recursiva para establecer la jerarquía zona-área.
- **Tarea**: Catálogo de tareas que se pueden realizar en cada área.
- **Fichaje**: Registro de entrada/salida de un empleado.
- **Visita**: Registro de cada área visitada durante un fichaje, con tiempo de permanencia.



Además, debido a las relaciones de tipo N:M, se incorporan dos entidades intermedias:

- LocalizacionTarea: Resuelve la relación N:M entre Localización y Tarea, indicando qué tareas están permitidas en cada zona o área.

- TareaRealizada: Une Visita con LocalizacionTarea y permite registrar qué tareas realizó el trabajador en cada estancia.

4.2. MODELO ESTÁNDAR DE DATOS

Las tablas de la base de datos SQLite tendrán los siguientes atributos:

Empleado (*idEmpleado*, nombre, dni)

Localizacion (*idLocalizacion*, nombre, latitud, longitud, radio, tipo, idZonaPadre)

Tarea *(idTarea*, nombreTarea, tipoTarea)

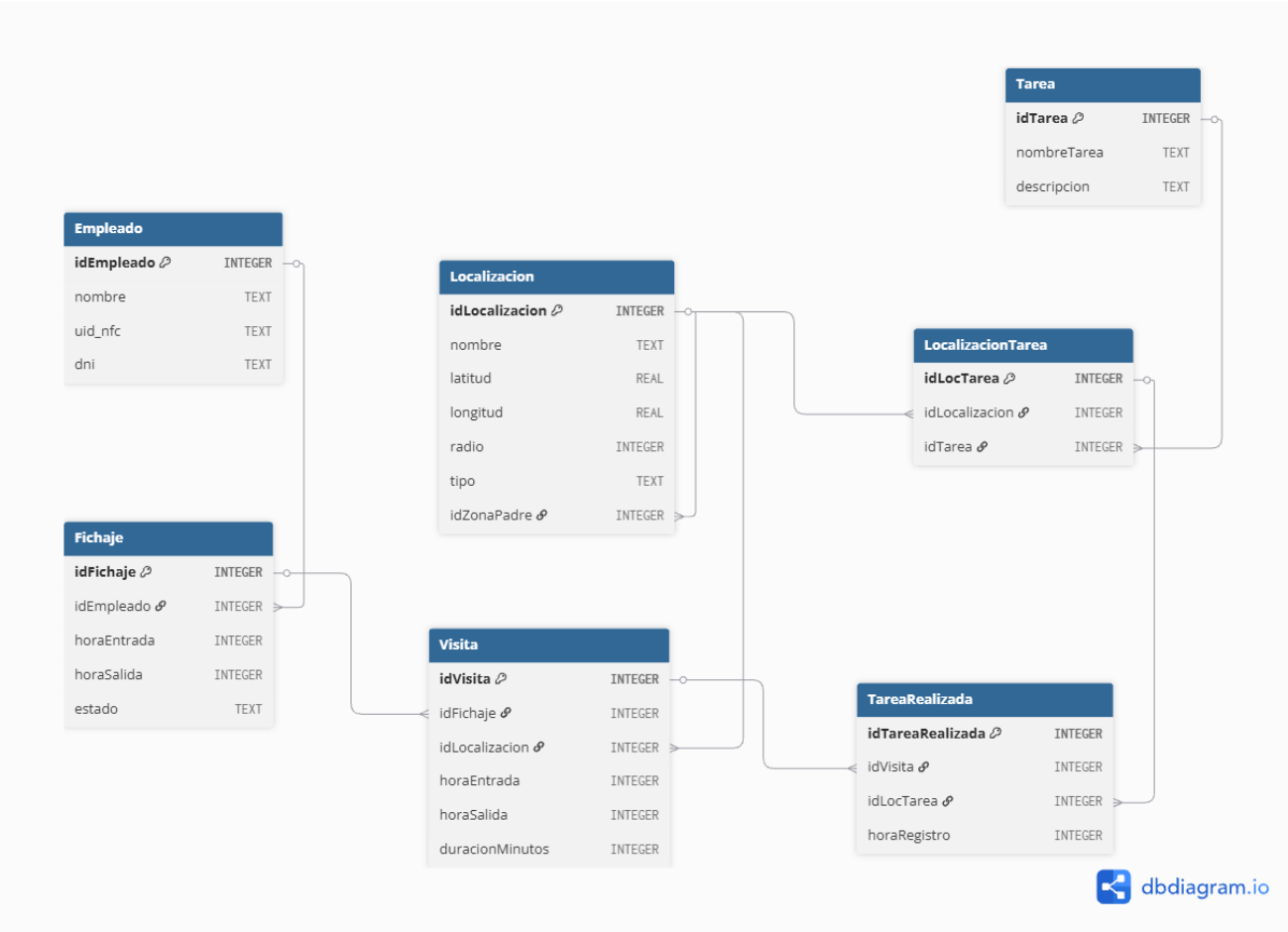
LocalizacionTarea *(LocalizacionTarea*, *idLocalizacion*, *idTarea)*

Fichaje *(idFichaje*, *idEmpleado*, horaEntrada, horaSalida, estado)

Visita *(idVisita*, *idFichaje*, *idLocalizacion*, horaEntrada, horaSalida, duracionMinutos)

TareaRealizada *(idTareaRealizada*, *idVisita*, *idLocTarea*, horaRegistro)

En la siguiente imagen podemos ver las tablas de una forma más sencilla con sus relaciones y sus atributos

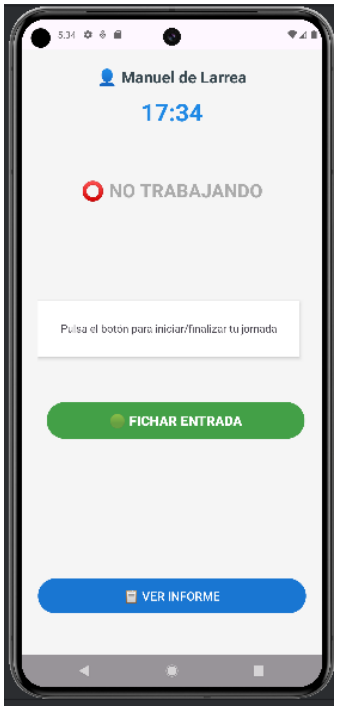


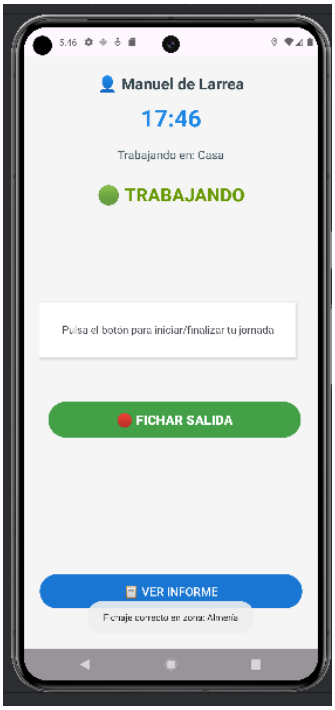
4.3. Diseño de la interfaz

La aplicación presentará una interfaz sencilla en la que el usuario pueda ver fácilmente los elementos importantes y que sea lo suficientemente clara como para no necesitar ninguna instrucción de uso.

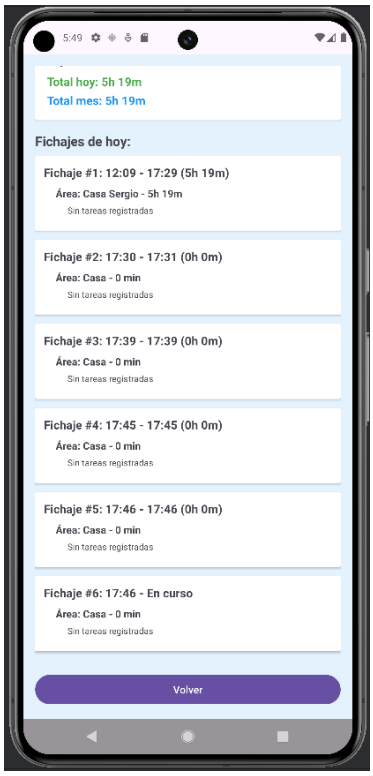
La pantalla de bienvenida pedirá al usuario su DNI, el cual al introducirlo se cotejará en la BBDD y en caso de corresponder a un registro real nos llevará a la pantalla principal de la aplicación, sino nos saldrá un mensaje de error.

En la pantalla principal se nos mostrará el usuario con una consulta a la BBDD seleccionando su nombre, la hora actual, el estado del fichaje (activo 'Trabajando', no activo 'No trabajando'), un textView que nos indica cómo fichar, el botón de fichar (que dentro tiene un círculo que cambia de verde a rojo según sea el fichaje de salida o entrada) y el botón de ver informe, el cual nos llevará a otra pantalla. Una vez que fichamos se nos actualizará la interfaz y nos mostrará el lugar en pantalla en el que estamos siendo "En camino a la zona de trabajo" o "Trabajando en: NombreÁrea "

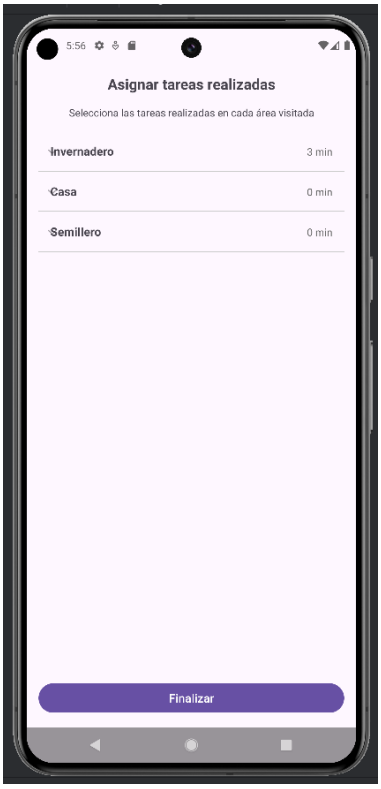


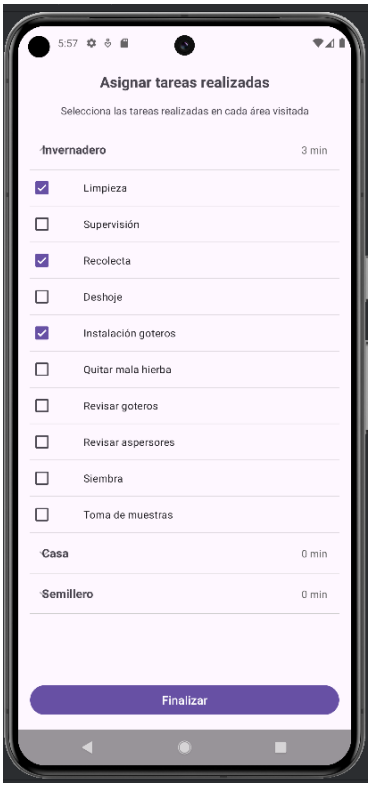


Luego podemos ver el informe que nos lleva a esta pantalla:

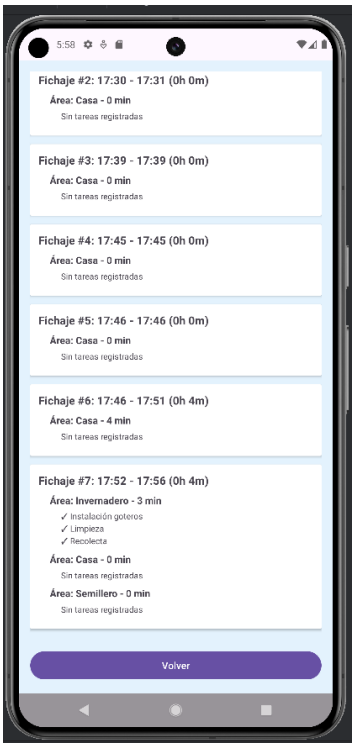


Y si pulsamos de nuevo en el botón de fichar para cerrar el fichaje:





Una vez seleccionadas las tareas se guardará el registro en la BBDD y podremos comprobarlo en el informe



4.4. TECNOLOGÍAS UTILIZADAS Y JUSTIFICACIÓN

Se ha elegido **Android Studio** como entorno de desarrollo porque considero que es un entorno sencillo y de fácil uso, que presenta facilidades como a la hora de crear interfaces con su editor visual, a parte al trabajar en la asignatura de Aplicaciones Móvil ya tenía una familiaridad adquirida. Otro aspecto a destacar es su emulador virtual el cual nos permite ir comprobando que funciona la aplicación en cada fase de desarrollo y actuar en consecuencia.

He optado por **Java** porque es el lenguaje que hemos estudiado en profundidad durante el ciclo. Esto me permite tener un mayor control sobre el código y defenderlo mejor en la exposición al entender cada línea escrita.

La base de datos se ha construido en SQLite usando la clase SQLiteOpenHelper. La clase DBHelper se encarga de crear y actualizar todas las tablas necesarias para la aplicación: empleados, localizaciones, tareas, relaciones entre localizaciones y tareas (LocalizacionTarea), fichajes diarios (Fichaje), visitas a áreas (Visita) y tareas realizadas (TareaRealizada). Los datos iniciales se insertan también desde DBHelper.

Para organizar el acceso a los datos, cada tabla cuenta con su propio repositorio con métodos para insertar, actualizar, borrar y consultar información. De este modo, aunque la creación de la base y de los datos iniciales está centralizada en DBHelper, la aplicación accede a los datos mediante repositorios, manteniendo la lógica de acceso separada de la interfaz y los servicios. Esta solución permite almacenar toda la información localmente sin depender de conexión a internet y nos deja preparados para una futura exportación de los datos a un server externo.

ESQUEMA DE LA APLICACIÓN: CLASES, ATRIBUTOS, MÉTODOS, RELACIONES, ETC

La aplicación sigue una arquitectura modular basada en Activities, Services, Repositorios, Entidades y una base de datos SQLite gestionada mediante SQLiteOpenHelper.

Activities

- **BienvenidaActivity:** Pantalla de inicio de sesión. Valida DNI en la base de datos.
- **MainActivity:** Pantalla principal. Gestiona el fichaje, muestra estado actual, inicia/detiene el servicio de tracking.
- **SeleccionTareasActivity:** Pantalla con ExpandableListView para seleccionar tareas realizadas en cada área.
- **InformeActivity:** Muestra el resumen de la jornada con las áreas visitadas y las tareas realizadas.

Servicios

- **TrackingService:** Foreground Service que verifica la ubicación cada 5 minutos, detecta cambios de área y registra visitas. Usa Handler con postDelayed para la ejecución periódica.

Base de datos

- **DBHelper (extiende SQLiteOpenHelper):** Creación de tablas, aplica actualizaciones y nos sirve para introducir los datos manualmente para las pruebas.

  Tablas principales en la base de datos:

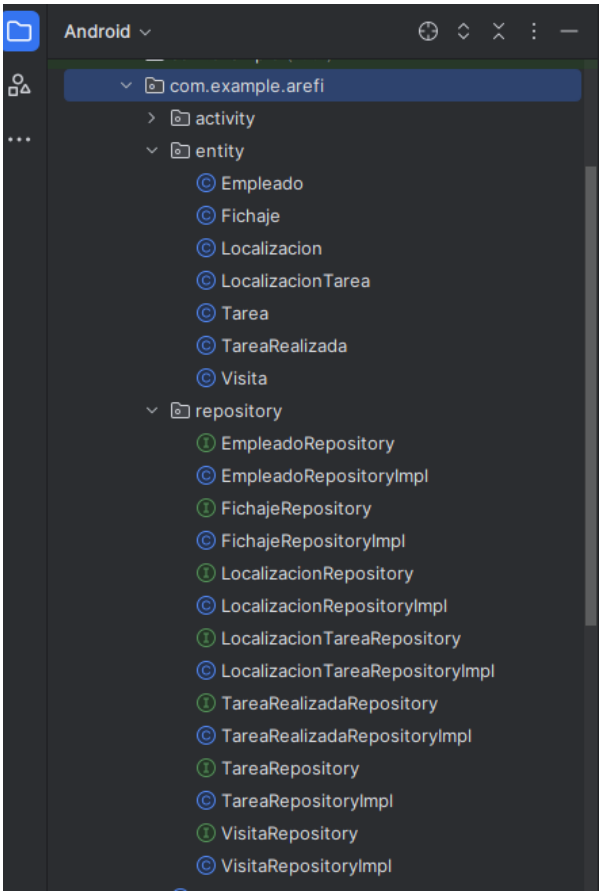
  - Fichaje: registro de entrada/salida del trabajador.
  - Visita: cada área visitada dentro de un fichaje.
  - Localizacion: lista de zonas y áreas.
  - Tarea: catálogo de tareas posibles.
  - LocalizacionTarea: tabla intermedia N–N entre áreas y tareas.
  - TareaRealizada: tareas hechas por visita.

Adaptadores

- **AdaptadorAreasExpandible:** Adaptador personalizado para el ExpandableListView. Gestiona la visualización de grupos (áreas) e hijos (tareas con checkboxes) y mantiene el estado de selección en un HashMap.

Repositories y Entidades

- Para cada Entidad hemos creado sus correspondientes Repositorios y sus implementaciones, quedándose la estructura de la aplicación de la siguiente manera



5. IMPLEMENTACIÓN

Damos paso a la explicación de cómo he llevado a cabo la elaboración del proyecto

5.1. ENTORNO DE DESARROLLO Y EJECUCIÓN

Tanto nuestro entorno de desarrollo como el de ejecución se componen de lo mismo: Android Studio, sus librerías integradas y su emulador para las pruebas, además de nuestro móvil personal junto con el de conocidos.

El desarrollo del proyecto ha ido de forma incremental, teniendo en cuenta que las pruebas se iban comprobando conforme se avanzaba el proyecto.

Planificación y definición de la base de datos

Comienzo el desarrollo teniendo claras las tablas con las que trabajaríamos, sus atributos y cómo se relacionarían, así como la idea general de uso de la aplicación.

Una vez definida y construida la base de datos, me dispuse a crear los CRUD de cada entidad, siguiendo la metodología que nos habían enseñado en acceso a datos.

Con la base y los métodos de acceso claros, pasamos al desarrollo de la interfaz de bienvenida, la identificación del usuario y su interacción con la base de datos.

Desarrollo de MainActivity y geolocalización

Tras esto, comencé a desarrollar la interfaz del MainActivity teniendo claro que no quería cargar mucho la pantalla y a dar las primeras funciones básicas como la geolocalización.

Para la geolocalización tuve que investigar bastante, mirando documentación oficial de Android y vídeos de YouTube para entender cómo funcionaban los permisos y cómo obtener las coordenadas del GPS.

Una vez conseguí que el móvil me devolviera la latitud y longitud, el siguiente paso fue validar que el empleado estuviera dentro de una zona de trabajo antes de permitirle fichar.

Luego siguió el registro de la creación del fichaje y el fichaje de salida, la actualización de la interfaz según el estado y que, si el usuario salía de la zona de trabajo, la aplicación automáticamente cerrase el fichaje (implementado posteriormente con el tracking).

Implementación del tracking automático

Después vino la parte del tracking automático, que fue lo que más me costó. Necesitaba que la app siguiera comprobando la ubicación aunque estuviera minimizada, y para eso tuve que aprender a usar los Foreground Services.

Al principio tenía el tracking dentro del propio MainActivity, con un Handler que cada cierto tiempo comprobaba la ubicación. El problema vino cuando minimizaba la app: el tracking se paraba. Android, para ahorrar batería, pausa o mata los procesos de las actividades que no están en primer plano.

Esto suponía un problema importante, ya que el trabajador no va a tener la app abierta todo el rato mientras trabaja. La app tenía que seguir detectando en qué área estaba, aunque el móvil estuviera en el bolsillo con la pantalla apagada.

Buscando soluciones, vi que Android tiene varios tipos de servicios para ejecutar tareas en segundo plano. Probé primero con un Service normal, pero no daba el resultado esperado.

La solución fue usar un Foreground Service, que es un servicio que muestra una notificación permanente mientras está activo. Al tener esa notificación visible, Android entiende que es algo importante para el usuario y no para el proceso.

Así que moví todo el código del tracking a una clase aparte llamada TrackingService. El MainActivity solo se encarga de iniciar el servicio cuando el empleado ficha entrada y pararlo cuando ficha salida. El servicio es el que hace el trabajo de comprobar la ubicación cada 5 minutos, detectar cambios de área y guardar las visitas en la base de datos.

Separarlo en una clase aparte dejó el código del MainActivity mucho más limpio y organizado. Cada clase tiene su responsabilidad: el MainActivity gestiona la interfaz y el TrackingService gestiona el seguimiento de ubicación.

Desarrollo de la selección de tareas

Con el tracking funcionando, me puse con la pantalla de selección de tareas. Quería que se vieran las áreas que había visitado el trabajador y dentro de cada una las tareas disponibles para marcar. Para esto usé un ExpandableListView que permite expandir y contraer grupos, quedando bastante visual.

Durante todo el proceso iba probando en el emulador, enviando coordenadas falsas para simular que estaba en una zona u otra. El Logcat me ayudó mucho para ir viendo qué pasaba por dentro, sobre todo cuando algo no funcionaba y no sabía por qué.

Ya al final probé en un móvil real para asegurarme de que el GPS de verdad funcionaba y no solo con coordenadas simuladas.

Problemas encontrados y soluciones

- **Actualización del área en MainActivity:**

  Al principio solo guardaba en el texto la zona de trabajo (idPadre), pero cuando quise cambiar al área, no conseguía que se actualizase junto con el tracking.
  El servicio detectaba bien que había cambiado de área y lo guardaba en la base de datos, pero la pantalla principal seguía mostrando el área antigua.
  Solución: usar un **BroadcastReceiver** para que el servicio de tracking envíe un mensaje cada vez que detecta un cambio de área y el MainActivity actualice la interfaz.

- **Estado de checkboxes en ExpandableListView:**

Cuando marcabas varias tareas en un área, al expandir otra área y volver a la primera, algunas tareas se habían desmarcado solas. Esto pasaba porque Android reutiliza las vistas para ahorrar memoria y, si no guardas bien el estado de los checkboxes, se pierden. Solución: usar un **HashMap** para almacenar qué tareas estaban marcadas y cuáles no, y consultarlo cada vez que se muestra la tarea.

- **Retrasos en el GPS:**

Cuando un usuario pulsaba fichar, en algunos móviles el GPS tardaba unos segundos en responder, provocando que la aplicación no reaccionara correctamente. Solución: añadir un **diálogo de carga** que bloquea la pantalla mientras se obtiene la ubicación, evitando que el usuario haga múltiples clics.

Se adjunta la dirección del código del proyecto en Github:

https://github.com/zulljin/manuelProyecto

6. PRUEBA

La gran mayoría de pruebas se han llevado a cabo mientras se avanzaba con la programación de la aplicación. La prueba final consistía en que todos los apartados y objetivos se cumplieran correctamente en un teléfono real (el mío concretamente).

Prueba	Procedimiento	Resultado
Login con DNI válido	Introducir DNI existente en la BD	Accede a MainActivity
Login con DNI inválido	Introducir DNI inexistente	Muestra error
Fichar dentro de zona	Enviar coordenadas dentro de una zona desde emulador	Crea fichaje correctamente
Fichar fuera de zona	Enviar coordenadas fuera de cualquier zona	Muestra mensaje de error
Detección de área	Fichar en coordenadas de un área específica	Detecta y muestra el área
Cambio de área	Cambiar coordenadas a otra área mientras hay fichaje activo	Crea nueva visita
Selección de tareas	Marcar tareas en ExpandableListView	Guarda selección
Servicio en segundo plano	Minimizar app y esperar 5 minutos	El tracking continúa

Los dispositivos que se han usado para las pruebas principalmente han sido:

- **Emulador:** Pixel 8 API 30 - Pruebas con coordenadas manuales
- **Dispositivo físico Xiaomi Redmi Note 2:** Pruebas con GPS real en ubicaciones configuradas

Una vez la app funcionaba correctamente en estos dispositivos, quise probarla en otros terminales diferentes. Para ello conté con la ayuda de dos conocidos que la instalaron en sus móviles y la probaron desde sus propias casas, configurando sus coordenadas como zonas de trabajo. Las pruebas incluyeron tanto intentos de fichaje sin la localización incluida en la base de datos como al contrario, para comprobar que la app respondía correctamente en cada caso.

El primer dispositivo fue un móvil de la marca Oppo y resultó ser el más problemático. El usuario pulsaba el botón de fichar y la app se quedaba en "Verificando ubicación..." sin llegar a hacer nada. Nunca conseguía obtener la ubicación. Investigando, descubrí que algunos fabricantes como Oppo, Xiaomi o Huawei tienen capas de personalización que gestionan la batería de forma muy agresiva y bloquean el acceso al GPS de las aplicaciones que no consideran prioritarias.

El segundo dispositivo fue un móvil con Android 14, un Honor. Al principio tenía el mismo problema de quedarse esperando, pero en su caso se solucionó añadiendo el diálogo de espera que bloquea la pantalla mientras se obtiene la ubicación. Una vez implementado esto, el usuario pudo fichar sin problemas.

Estas pruebas en dispositivos reales fueron muy útiles porque sacaron a la luz problemas que en el emulador o en mi terminal no aparecían.

7. MANTENIMIENTO

Antes de comenzar la etapa de mantenimiento primero debemos zanjar varios aspectos importantes: mejora de la interfaz, la creación del usuario administrador para gestionar los empleados, la conexión a una BBDD externa a la aplicación que permita sincronizar los datos de la aplicación con los del servidor o funcionalidades nuevas como habilitar el fichaje con el NFC del teléfono.

Una vez resuelto lo anterior daríamos comienzo a la etapa del mantenimiento.

Como la idea es vender nuestra aplicación a varias empresas lo más óptimo será crear una base de datos en nuestro servidor para cada empresa. Cuando contratemos a una nueva empresa seremos los encargados de la creación de la nueva base de datos y la creación de sus tablas.

Los datos recopilados de esas empresas se podrán exportar a CSV (Excel) mediante la creación de ETLs (Extracción, Transformación y Carga) en Visual Studio que nos permitirá automatizar los datos obtenidos.

Será importante también, revisar regularmente que la información enviada al servidor sea correcta. Para facilitar esto, durante la creación de la ETL se pueden añadir scripts que alerten automáticamente cuando algo no concuerde.

Cada cierto tiempo tendremos que actualizar la aplicación, generando mejoras en el código o simplemente actualizándolo por avances de Android o permisos nuevos.

Por último y algo totalmente necesario es el soporte a los clientes. Tendremos que atender cualquier incidencia que haya sido reportada, tomando dicha incidencia también para mejorar el código dentro de lo posible.

8.CONCLUSIONES

La aplicación valida la ubicación del empleado antes de permitir el fichaje, registra automáticamente las áreas visitadas durante la jornada y permite seleccionar y registrar las tareas realizadas en la base de datos. Así que podemos decir que el desarrollo de la aplicación ha cumplido con los objetivos iniciales.

Durante el desarrollo se han aplicado conocimientos adquiridos en el grado, especialmente en las asignaturas de Programación en Tecnologías Móviles, Acceso a Datos y Programación de Servicios y Procesos. He podido profundizar en conceptos como los permisos en tiempo de ejecución de Android, los servicios en segundo plano o la comunicación entre componentes mediante Broadcast.

Aunque la idea inicial de incluir tanto NFC como Geolocalización para el fichaje tuvo que ser descartada al comienzo de la creación del proyecto he de decir que estoy muy satisfecho con el resultado final de la aplicación. Todavía queda trabajo por delante para mejorarla, pero realmente le veo una utilidad real para empresas con extensiones amplias de terreno donde la aplicación puede facilitar el día a día tanto de administradores como de empleados.

9.BIBLIOGRAFIA

Documentación oficial

- Android Developers - Location: developer.android.com/develop/sensors-and-location/location
- Android Developers - SQLite: developer.android.com/training/data-storage/sqlite
- Android Developers - Foreground Services: developer.android.com/develop/background-work/services/foreground-services
- Android Developers - Permissions: developer.android.com/training/permissions/requesting

Tutoriales y recursos en español

- Develou - Bases de datos SQLite en Android: develou.com/android-sqlite-bases-de-datos
- Academia Android - Tutoriales: academiaandroid.com
- SGoliver - Curso Android: sgoliver.net/blog/curso-de-programacion-android
- Código Facilito - Android: codigofacilito.com/cursos/android

Recursos técnicos consultados

- Stack Overflow: stackoverflow.com (resolución de problemas específicos)
- GeeksforGeeks - Android Services: geeksforgeeks.org/services-in-android